Local VLM Camera Intelligence Platform — Technical Scope

1. Objective

Build a local system that can:

1. Ingest recorded videos and live camera streams.
2. Avoid permanently storing unchanged footage.
3. Divide active footage into searchable time windows.
4. Generate visual descriptions and vector embeddings.
5. Let operators query individual or multiple cameras using natural language.
6. Combine results occurring across multiple clips and timeslots.
7. Let users create custom triggers assigned to selected cameras.
8. Report trigger matches with supporting video evidence.
9. Keep camera footage and AI processing on the local network.

2. Initial MVP boundaries

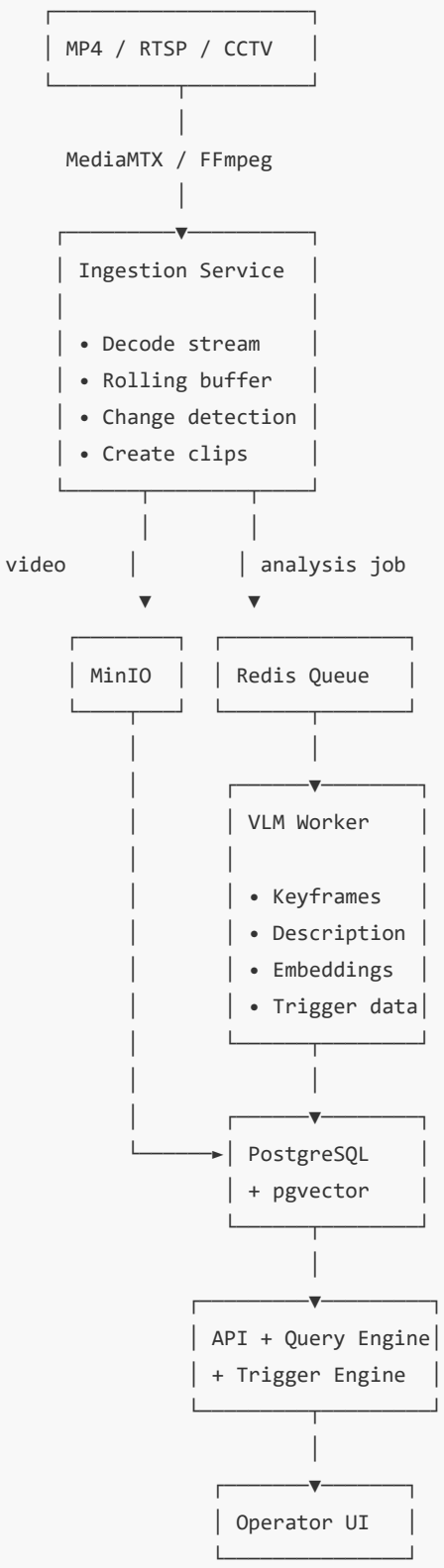
The first version will support:

- Local MP4 files
- Simulated RTSP live streams
- Real RTSP/ONVIF CCTV cameras later
- User-defined camera names
- Activity/change-based video retention
- Natural-language historical search
- Custom live triggers
- Evidence playlists across multiple clips
- Local VLM inference
- Local database and object storage

The first version will not promise:

- Facial recognition
- Identification of a person across different cameras
- Exact long-term person tracking
- Guaranteed interpretation of intent
- Reliable audio understanding
- Automatic connection to every proprietary CCTV recorder

3. Local architecture



4. Recommended local technology stack

Component	Recommended technology	Purpose

Database	PostgreSQL with pgvector	Metadata, triggers, events and vectors
Video storage	MinIO	Local S3-compatible video storage
Temporary queue	Redis	Processing jobs and live event delivery
RTSP gateway	MediaMTX	Simulated and real RTSP streams
Video processing	FFmpeg initially	Decode, segment and extract frames
Backend API	Python FastAPI	Cameras, search, triggers and playback
Database access	SQLAlchemy + Alembic	Models and migrations
Background workers	Celery, Dramatiq or custom Redis workers	VLM and embedding jobs
VLM serving	vLLM on NVIDIA Linux	Local video/image-language inference
Development VLM	Smaller model through Transformers or MLX	Laptop development
Frontend	React/Next.js	Operator interface
Live updates	WebSockets	Events and processing status
Local packaging	Docker Compose	Repeatable one-machine installation

Redis can initially be omitted in a single-camera prototype, but it becomes useful once ingestion and AI processing run independently.

5. Database and storage setup

5.1 Storage responsibilities

PostgreSQL stores:

- Camera configuration
- Segment timestamps

- Object-storage paths
- VLM observations
- Vector embeddings
- Trigger definitions
- Trigger matches
- Query history
- Evidence playlists

MinIO stores:

- Original active video segments
- Keyframe images
- Event thumbnails
- Exported evidence videos

Temporary local disk stores:

- The rolling camera buffer
- Incomplete fragments
- Frames waiting for analysis

Video binaries should not be stored directly inside PostgreSQL.

5.2 Core database tables

`cameras`

```
id
display_name
source_type
source_uri_encrypted
enabled
analysis_fps
retention_days
created_at
```

The user chooses `display_name`. The system should not invent physical meanings such as “North Gate.”

`video_segments`

```
id
camera_id
start_time
end_time
duration_ms
storage_uri
activity_score
status
codec
width
```

```
height
created_at
```

A segment represents one stored interval of active footage.

`static_periods`

```
id
camera_id
start_time
end_time
reference_frame_uri
health_status
```

This records that the camera remained operational but its scene did not materially change.

`analysis_windows`

```
id
segment_id
camera_id
start_time
end_time
vlm_description
structured_observation_json
vlm_model
confidence
processing_status
```

An analysis window is normally 10–20 seconds and can overlap the next window.

`keyframes`

```
id
analysis_window_id
timestamp
storage_uri
visual_embedding
```

`text_embeddings`

```
id
analysis_window_id
embedding_type
embedding
source_text
```

`triggers`

```
id
name
natural_language_rule
compiled_rule_json
minimum_confidence
cooldown_seconds
pre_roll_seconds
post_roll_seconds
enabled
created_at
```

`trigger_cameras`

```
trigger_id
camera_id
```

This many-to-many table allows every trigger to be assigned to one or more user-selected cameras.

`events`

```
id
trigger_id
camera_id
start_time
end_time
confidence
summary
status
created_at
```

`event_evidence`

```
id
event_id
segment_id
start_offset_ms
end_offset_ms
evidence_reason
sequence_order
```

`query_sessions`

```
id
query_text
camera_filter
start_filter
end_filter
```

```
created_at
```

`query_results`

```
id
query_session_id
analysis_window_id
vector_score
rerank_score
verification_score
verified
sequence_group_id
```

5.3 Suggested pgvector indexes

Use separate embeddings for:

- Visual keyframes
- Factual text descriptions

Start with HNSW indexes. Keep ordinary indexes on:

- `camera_id`
- `start_time`
- `end_time`
- `trigger_id`
- `processing_status`

Camera and time filters should be applied before or alongside vector ranking.

6. Video sources for development

Stage 1: Local MP4 files

Start with manually selected MP4 files containing controlled scenarios:

- A person entering and leaving
- A bag being placed and later removed
- Several people crossing the scene
- A vehicle arriving and departing
- Long periods with no movement
- Similar-looking activity that should not trigger an event

Create some test videos yourself so the expected answers and exact timestamps are known.

Recommended minimum test set:

```
test-data/
  camera-01/
    person-enters.mp4
```

```
    bag-left-behind.mp4
    static-room.mp4
  camera-02/
    vehicle-arrival.mp4
    two-people-crossing.mp4
```

These files can first be processed as completed recordings.

Stage 2: Simulated live RTSP

Use MediaMTX and FFmpeg to loop an MP4 as though it were a CCTV camera:

```
MP4 file → FFmpeg loop → MediaMTX → RTSP URL → ingestion service
```

Example logical sources:

```
rtsp://mediamtx:8554/cam-01
rtsp://mediamtx:8554/cam-02
```

This tests:

- Reconnection
- Continuous ingestion
- Rolling buffers
- Live triggers
- Multiple concurrent cameras
- Stream interruptions

Stage 3: Public evaluation footage

Use public surveillance-style datasets only for evaluation and demonstration. Choose videos with clear licences and annotations.

Useful categories include:

- Stationary-camera pedestrian scenes
- Parking areas
- Object placement/removal
- Vehicle movement
- Entry/exit activity

Public footage should supplement controlled test videos, not replace them. Many datasets do not contain the precise trigger scenarios required by the product.

Stage 4: Live non-production camera

Connect one:

- USB webcam
- IP camera

- Phone streaming application
- Existing RTSP-capable test camera

The first real camera should be on an isolated test network without sensitive public footage.

Stage 5: CCTV/NVR integration

Support these connection methods in order:

1. Direct RTSP camera URL
2. RTSP URL supplied by an NVR
3. ONVIF discovery followed by RTSP
4. Vendor-specific integration only when necessary

7. Video ingestion flow

7.1 Stream handling

For every camera:

1. Connect to its RTSP stream.
2. Read the low-resolution substream for analysis when available.
3. Keep a 30–60 second rolling buffer.
4. Sample approximately one frame per second for scene-change analysis.
5. Measure visual change against the recent stable background.
6. Start an active interval when change remains above the threshold.
7. Include footage from before the change using the rolling buffer.
8. End the interval after the scene remains stable.
9. Add post-event footage.
10. Write the completed segment to MinIO.
11. Create a `video_segments` database record.
12. Submit the segment for AI analysis.

7.2 Initial configuration

```
Analysis sampling:      1 frame per second
Rolling buffer:         60 seconds
Pre-roll:               10 seconds
Post-roll:              10 seconds
Storage fragments:      4–5 seconds
Analysis window:        12 seconds
Window overlap:         2–4 seconds
Stable timeout:         10 seconds
Health snapshot:        Every 10 minutes
```

These values should be configuration, not hard-coded behavior.

7.3 Change detection

The first version can use:

- Resized grayscale frames
- Structural similarity
- Pixel difference
- Perceptual hashes
- A configurable region mask

This is not object detection. It only decides whether the scene changed enough to justify storing and analysing the footage.

Changes caused by rain, trees, displays or lighting will require camera-specific masks and thresholds.

8. VLM analysis

8.1 Model approach

Use a replaceable model interface so the rest of the system is not tied to one model.

Initial model profile:

- Small local VLM for development
- Qwen-family video-capable VLM for the pilot
- Quantized 7B–8B-class model on an NVIDIA GPU
- Larger model or hosted verification service only if accuracy requires it

The VLM receives:

- A sequence of sampled frames
- Relative frame timestamps
- Camera ID
- A strict factual prompt
- A structured JSON response schema

Example response:

```
{
  "summary": "A person enters, places a dark bag near the wall and leaves.",
  "entities": [
    {
      "temporary_id": "person_1",
      "type": "person"
    },
    {
      "temporary_id": "object_1",
      "type": "bag"
    }
  ],
  "actions": [
    {
```

```
      "type": "enter",
      "subject": "person_1",
      "start_second": 1.2,
      "end_second": 2.8
    },
    {
      "type": "place_object",
      "subject": "person_1",
      "object": "object_1",
      "start_second": 5.1,
      "end_second": 7.4
    },
    {
      "type": "leave",
      "subject": "person_1",
      "start_second": 9.0,
      "end_second": 11.1
    }
  ],
  "visible_objects": ["person", "bag"],
  "uncertainties": [],
  "overall_confidence": 0.88
}
```

Temporary IDs apply only within the clip. They must not imply persistent identity across cameras.

9. Vectorisation flow

9.1 What gets vectorised

For each analysis window:

1. Extract three to six representative keyframes.
2. Generate one visual embedding per keyframe.
3. Generate a factual VLM description.
4. Generate a text embedding for the description.
5. Store structured entities and actions separately.
6. Store all timestamps and storage references.

Do not create an embedding for every original video frame. That would create unnecessary storage and duplicate information.

9.2 Recommended vector layers

Use two complementary indexes:

Visual index

Input:

- Keyframe image

Purpose:

- Visual appearance queries
- Object or scene similarity
- Queries where the text description omitted a visible detail

Text index

Input:

- VLM-generated factual description

Purpose:

- Actions
- Relationships
- Natural-language event queries
- Operational descriptions

The query engine searches both and combines their scores.

9.3 Preventing missed cross-window events

Use overlapping analysis windows:

```
Window A: 00:00–00:12
Window B: 00:09–00:21
Window C: 00:18–00:30
```

This prevents an action beginning near the end of one window from being separated entirely from its outcome.

10. Historical query flow

Example query:

> *Show where someone placed an object and later left without it on CAM-02 yesterday.*

Processing steps

1. Parse camera and time filters.
2. Convert the semantic portion into embeddings.
3. Search text and visual vectors inside the selected filters.
4. Retrieve approximately 30–50 candidate windows.
5. Combine vector scores with:
    - Keyword matches
    - Structured action matches
    - Time relevance
    - Existing event confidence
1. Rerank the best candidates.
2. Expand each candidate to adjacent video windows.

3. Merge overlapping or closely adjacent intervals.
4. Send only the best sequences to the VLM for final verification.
5. Remove unsupported candidates.
6. Generate a playback manifest.
7. Return the first results to the interface.

Query response stages

The interface should show:

```
Searching indexed footage…
Found 8 possible matches…
Verifying 3 sequences…
2 verified results
```

Candidates can appear quickly, but they must be labelled “possible match” until verification completes.

11. Combining multiple frames and timeslots

Results should be grouped using timestamps, not vector similarity alone.

Example:

```
CAM-02
14:20:51–14:21:12  Person arrives
14:21:09–14:21:31  Object is placed
14:21:30–14:21:48  Person leaves
```

Because these windows overlap, merge them into:

```
CAM-02 — 14:20:51–14:21:48
```

If the intervals contain a substantial gap:

```
CAM-02 — 14:20:51–14:21:48
5 minutes 12 seconds not shown
CAM-02 — 14:27:00–14:27:29
```

The operator should see an evidence playlist with an explicit gap marker. The system must not make separated footage appear continuous.

The playback manifest can contain:

```
{
  "result_id": "result-181",
  "parts": [
    {
      "camera_id": "CAM-02",
```

```
      "video_uri": "segment-100.mp4",
      "source_start_ms": 12000,
      "source_end_ms": 48000
    },
    {
      "type": "gap",
      "duration_ms": 312000
    },
    {
      "camera_id": "CAM-02",
      "video_uri": "segment-108.mp4",
      "source_start_ms": 0,
      "source_end_ms": 29000
    }
  ]
}
```

Create a physical combined MP4 only when the operator requests an export.

12. Custom trigger flow

12.1 Trigger creation

The user:

1. Enters a natural-language condition.
2. Selects one or more cameras.
3. Configures confidence, cooldown and evidence duration.
4. Tests the rule against sample footage.
5. Enables the trigger.

Example:

```
Name: Object left behind
Rule: Report when someone places an object and leaves it behind
Cameras: CAM-01, CAM-04
Confidence: 80%
Cooldown: 2 minutes
Evidence: 10 seconds before and 20 seconds after
```

12.2 Trigger compilation

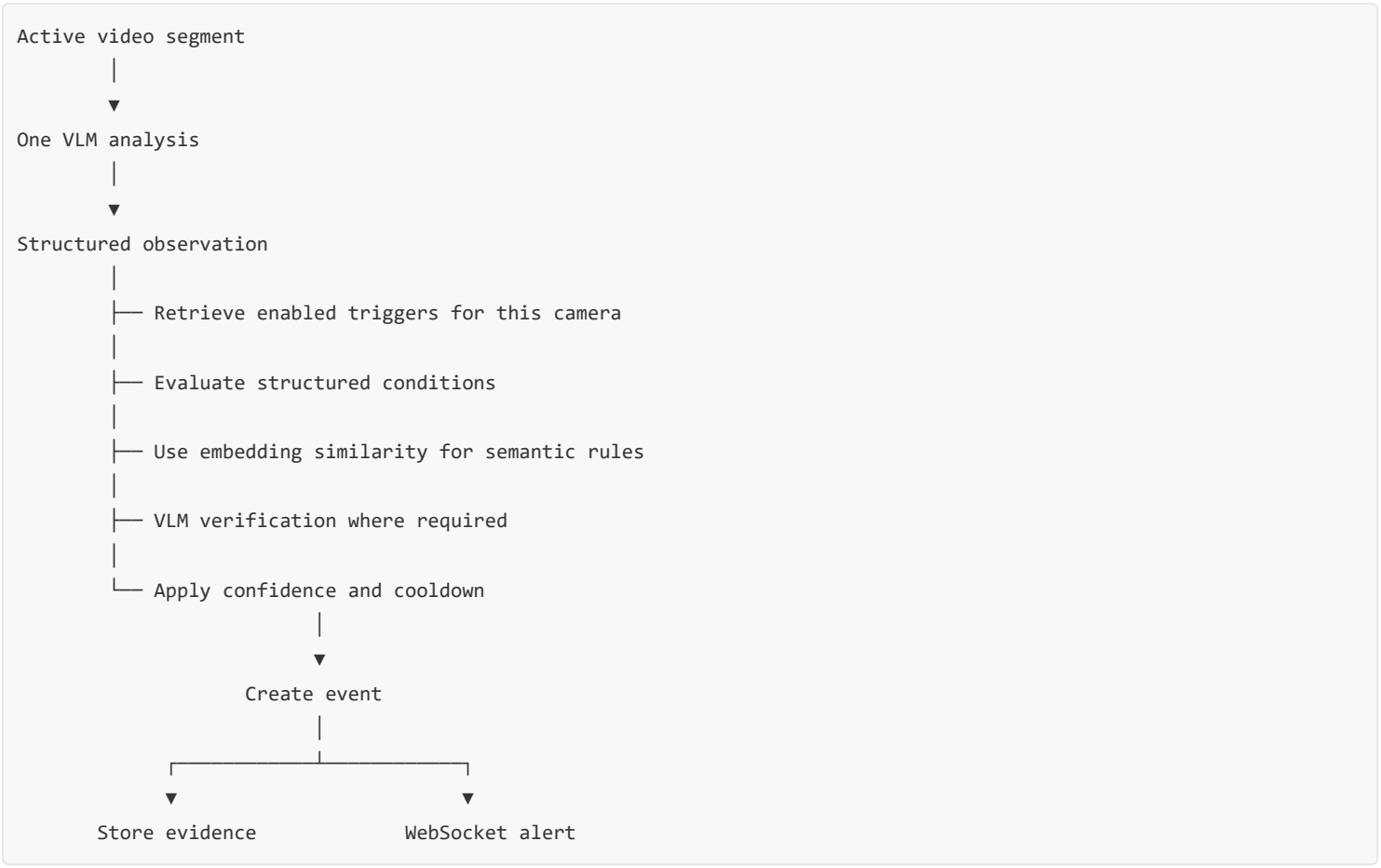
The trigger service converts the rule into structured criteria:

```
{
  "required_entities": ["person", "portable_object"],
  "required_action_sequence": [
    "person_places_object",
    "person_moves_away",
    "object_remains"
```

```
  ],
  "maximum_sequence_gap_seconds": 30,
  "requires_vlm_verification": true
}
```

The original natural-language rule must also be retained.

12.3 Live evaluation



A clip is analysed once. The resulting observation is evaluated against every applicable trigger. The VLM should not be called separately for every trigger unless final semantic verification is genuinely necessary.

12.4 Historical trigger testing

Before enabling a trigger, allow it to run against:

- A selected test video
- The previous hour
- The previous 24 hours
- A custom historical range

Historical testing uses vector retrieval to find candidates and VLM verification to estimate false positives.

13. Backend API scope

Minimum endpoints:

```
POST   /cameras
GET    /cameras
PATCH  /cameras/{id}
POST   /cameras/{id}/test-connection

POST   /videos/upload
GET    /segments
GET    /segments/{id}/playback

POST   /queries
GET    /queries/{id}
GET    /queries/{id}/results

POST   /triggers
GET    /triggers
PATCH  /triggers/{id}
POST   /triggers/{id}/test
POST   /triggers/{id}/enable
POST   /triggers/{id}/disable

GET    /events
GET    /events/{id}
POST   /events/{id}/acknowledge
POST   /events/{id}/export

WS     /live/events
WS     /queries/{id}/progress
```

14. Suggested local project structure

```
sentinel/
  docker-compose.yml
  .env.example

  services/
    api/
    ingestion/
    analysis/
    trigger-engine/
    query-engine/

  shared/
    database/
    schemas/
    model-adapters/
    storage/
    video/

  frontend/
```

```
  infrastructure/
    postgres/
    mediamtx/
    minio/

  test-data/
  tests/
    ingestion/
    search/
    triggers/
    end-to-end/
```

15. Implementation phases

Phase 1 — Storage foundation

Deliver:

- Docker Compose
- PostgreSQL with pgvector
- MinIO
- Redis
- Database migrations
- Camera and segment APIs
- Storage retention configuration

Acceptance test:

- Upload one video.
- Store it in MinIO.
- Retrieve its metadata from PostgreSQL.
- Play it through the API.

Phase 2 — Video ingestion

Deliver:

- MP4 ingestion
- RTSP ingestion
- Rolling buffer
- Change detection
- Active-segment storage
- Static-period records

Acceptance test:

- Loop a test MP4 as RTSP.
- Store only the active interval.
- Include pre-roll and post-roll.

- Recover after an interrupted stream.

Phase 3 — VLM and vector pipeline

Deliver:

- Keyframe extraction
- VLM structured observations
- Visual embeddings
- Text embeddings
- pgvector indexes
- Processing status and retry handling

Acceptance test:

- Process a known test video.
- Confirm expected entities/actions.
- Retrieve the relevant interval using a natural-language query.

Phase 4 — Historical queries

Deliver:

- Camera and time filters
- Hybrid vector search
- Candidate reranking
- VLM verification
- Interval expansion and merging
- Evidence playlists

Acceptance test:

- Query an event spanning overlapping windows.
- Return one merged result.
- Display gaps explicitly when results are separated.

Phase 5 — Custom triggers

Deliver:

- Trigger builder
- Camera assignment
- Trigger compilation
- Live evaluation
- Cooldown and duplicate suppression
- Event notification
- Historical trigger testing

Acceptance test:

- Assign a trigger to CAM-01 only.
- Confirm CAM-02 never evaluates it.
- Trigger an event on CAM-01.
- Receive a live notification and playable evidence.

Phase 6 — CCTV pilot

Deliver:

- Direct RTSP configuration
- Encrypted camera credentials
- ONVIF discovery where supported
- Camera health monitoring
- Retention cleanup
- Operational logging

Acceptance test:

- Run one real camera continuously.
- Recover from network loss.
- Search historical footage.
- Receive a verified custom-trigger event.

16. Initial deployment target

Recommended pilot:

```
Cameras:             1–4
Analysis rate:       1 FPS
Stored footage:      Activity-only
VLM window:          12 seconds
Concurrent workers:  1–2
GPU:                 NVIDIA GPU with 16–24 GB VRAM
System RAM:          32 GB
Storage:             1 TB SSD
Retention:           7–30 days
```

A CPU-only system is suitable for functional development but will probably not provide useful real-time VLM performance.

17. Definition of “real time”

For the initial product, real time should mean:

- Activity detected within approximately 1–2 seconds.
- Clip analysed after sufficient evidence becomes available.
- Trigger reported within approximately 5–20 seconds of the relevant action.
- Candidate historical results returned quickly.

- Verified historical results added progressively.

It should not claim sub-second detection while relying on 10–20 second video windows.

18. First implementation milestone

The first complete vertical slice should be:

```
One MP4
  → simulated RTSP stream
  → activity-based segment
  → MinIO storage
  → PostgreSQL metadata
  → VLM observation
  → text + visual embeddings
  → natural-language query
  → merged evidence playback
  → one custom trigger
  → stored event and UI notification
```

This proves the complete architecture before adding additional cameras, advanced tracking or production CCTV integrations.